LAPORAN PRAKTIKUM

STRUKTUR DATA

QUEUE

PEKAN 4

Disusun Oleh :

KINAYA NOVRYA MANDA

(2511531016)

Dosen Pengampu :

DR. WAHYUDI, S.T, M.T

Asisten Praktikum :

MUHAMMAD GALID AVERO

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis sampaikan kehadirat Allah SWT. Salawat dan salam disampaikan kepada Nabi Muhammad SAW. Karena thaufik dan hidayah-Nya, laporan praktikum ini dapat diselesaikan dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban dari kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk memperdalam pemahaman mengenai konsep dasar serta penerapan bahasa pemrograman Java.

Melalui praktikum ini, penulis memperoleh pengalaman langsung dalam memahami Implementasi Queue menggunakan Java. Diharapkan laporan ini dapat memberikan gambaran yang jelas mengenai materi yang telah dipelajari serta hasil dari percobaan yang dilakukan selama praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi isi maupun penyajiannya. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang.

Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu, asisten laboratorium, serta semua pihak yang telah memberikan bimbingan, arahan, dan dukungan sehingga laporan praktikum ini dapat tersusun. Semoga laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami dasar-dasar pemrograman Java.

Padang, April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....i

DAFTAR ISI.....ii

DAFTAR LAMPIRAN.....iii

BAB 1 PENDAHULUAN

1.1 Latar Belakang..... 1

1.2 Tujuan Praktikum.....2

1.3 Manfaat Praktikum.....2

BAB 2 PEMBAHASAN

2.1 Dasar Teori.....3

2.2 Kode Program..... 5

2.3 Langkah Kerja.....9

2.4 Analisis Hasil..... 21

BAB 3 KESIMPULAN

3.1 Hasil Praktikum.....22

3.2 Saran Pengembangan..... 25

DAFTAR KEPUSTAKAAN.....26

TUGAS PRAKTIKUM PEKAN 4.....27

DAFTAR LAMPIRAN

Kode Program 2.1 (Kode Program QueueArray)..... 6

Kode Program 2.2 (Kode Program QueueArrayDriver).....6

Kode Program 2.3 (Kode Program QueueLinkedList)..... 7

Kode Program 2.4 (Kode Program ReverseData).....7

Kode Program 2.5 (Kode Program IterasiQueue).....8

Gambar 2.1 (Langkah 1 QueueArray)..... 9

Gambar 2.2 (Langkah 2 QueueArray)..... 9

Gambar 2.3 (Langkah 3 QueueArray)..... 10

Gambar 2.4 (Langkah 4 QueueArray)..... 10

Gambar 2.5 (Langkah 5 QueueArray)..... 10

Gambar 2.6 (Langkah 6 QueueArray)..... 10

Gambar 2.7 (Langkah 7 QueueArray)..... 10

Gambar 2.8 (Langkah 8 QueueArray)..... 11

Gambar 2.9 (Langkah 9 QueueArray)..... 11

Gambar 2.10 (Langkah 10 QueueArray)..... 11

Gambar 2.11 (Langkah 11 QueueArray).....12

Gambar 2.12 (Langkah 12 QueueArray)..... 12

Gambar 2.13 (Langkah 13 QueueArray)..... 12

Gambar 2.14 (Langkah 14 QueueArray)..... 12

Gambar 2.15 (Langkah 1 QueueArrayDriver).....13

Gambar 2.16 (Langkah 2 QueueArrayDriver).....13

Gambar 2.17 (Langkah 3 QueueArrayDriver).....13

Gambar 2.18 (Langkah 4 QueueArrayDriver).....14

Gambar 2.19 (Langkah 5 QueueArrayDriver)..... 14
Gambar 2.20 (Langkah 1 QueueLinkedList)..... 15
Gambar 2.21 (Langkah 2 QueueLinkedList)..... 15
Gambar 2.22 (Langkah 3 QueueLinkedList)..... 15
Gambar 2.23 (Langkah 4 QueueLinkedList)..... 16
Gambar 2.24 (Langkah 5 QueueLinkedList)..... 16
Gambar 2.25 (Langkah 6 QueueLinkedList)..... 16
Gambar 2.26 (Langkah 1 ReverseData).....17
Gambar 2.27 (Langkah 2 ReverseData).....17
Gambar 2.28 (Langkah 3 ReverseData).....17
Gambar 2.29 (Langkah 4 ReverseData).....18
Gambar 2.30 (Langkah 5 ReverseData).....18
Gambar 2.31 (Langkah 6 ReverseData).....18
Gambar 2.32 (Langkah 1 IterasiQueue).....19
Gambar 2.33 (Langkah 2 IterasiQueue).....19
Gambar 2.34 (Langkah 3 IterasiQueue).....19
Gambar 2.35 (Langkah 4 IterasiQueue).....20
Gambar 2.36 (Langkah 5 IterasiQueue).....20
Gambar 2.37 (Langkah 6 IterasiQueue).....20
Gambar 3.1 (Hasil Praktikum QueueArrayDriver).....22
Gambar 3.2 (Hasil Praktikum QueueLinkedList).....23
Gambar 3.3 (Hasil Praktikum ReverseData).....23
Gambar 3.4 (Hasil Praktikum IterasiQueue).....24

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu konsep dasar dalam pemrograman yang berperan penting dalam mengatur dan mengelola data secara efisien. Salah satu struktur data yang sering digunakan adalah queue (antrian), yang menerapkan prinsip FIFO (First In First Out), di mana data yang pertama masuk akan menjadi yang pertama keluar. Konsep ini banyak dijumpai dalam kehidupan sehari-hari, seperti antrian di loket atau sistem pelayanan.

Dalam pemrograman Java, implementasi queue dapat dilakukan dengan berbagai cara, seperti menggunakan array maupun struktur data bawaan seperti LinkedList. Masing-masing pendekatan memiliki karakteristik dan cara kerja yang berbeda, sehingga penting untuk dipahami agar dapat memilih metode yang sesuai dengan kebutuhan. Selain itu, pemahaman tentang operasi dasar seperti enqueue, dequeue, serta cara menampilkan dan mengakses elemen juga menjadi bagian penting dalam penggunaan queue.

Melalui praktikum ini, dilakukan beberapa percobaan yang berkaitan dengan queue, mulai dari implementasi sederhana menggunakan array, penggunaan LinkedList, hingga operasi tambahan seperti membalik urutan data dan melakukan iterasi. Dengan melakukan percobaan secara langsung, pemahaman terhadap konsep queue diharapkan menjadi lebih jelas dan tidak hanya sebatas teori.

1.2 Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami konsep dasar struktur data queue beserta prinsip kerjanya (FIFO).
2. Mampu mengimplementasikan queue menggunakan array dan LinkedList dalam bahasa Java.
3. Mengetahui dan mempraktikkan operasi-operasi dasar queue seperti enqueue, dequeue, serta iterasi dan manipulasi data.

1.3 Manfaat Praktikum

Manfaat dilakukannya praktikum ini adalah sebagai berikut :

1. Membantu mahasiswa memahami penerapan nyata struktur data queue dalam pemrograman.
2. Melatih kemampuan dalam mengimplementasikan dan mengelola data menggunakan berbagai metode queue.
3. Memberikan pengalaman praktis sehingga mahasiswa lebih siap dalam menyelesaikan permasalahan yang berkaitan dengan struktur data di dunia nyata.

BAB II
PEMBAHASAN

2.1 Dasar Teori

Struktur Data Queue (Antrian) adalah konsep dasar dalam ilmu komputer yang digunakan untuk menyimpan dan mengelola data dalam urutan tertentu. Antrian mengikuti prinsip *"First in, First out"* (FIFO), di mana elemen pertama yang ditambahkan ke dalam queue adalah elemen pertama yang dihapus. Antrian biasanya digunakan dalam berbagai algoritma dan aplikasi karena kesederhanaan dan efisiensinya dalam mengelola aliran data.

Antrian beroperasi seperti garis di mana elemen ditambahkan pada satu ujung (rear) dan dihapus dari ujung lainnya (front). Bayangkan sebuah antrian seperti orang-orang yang mengantre di supermarket. Orang pertama yang mengantri adalah orang pertama yang dapat membayar dan meninggalkan supermarket.

Berdasarkan jenisnya, queue dibedakan menjadi:

1. Queue ukuran tetap, yaitu queue yang memiliki kapasitas tetap dan tidak dapat bertambah atau berkurang, sehingga dapat mengalami kondisi overflow saat penuh dan underflow saat kosong. Biasanya diimplementasikan menggunakan array.
2. Queue ukuran dinamis, yaitu queue yang ukurannya dapat berubah secara otomatis sesuai kebutuhan, biasanya diimplementasikan menggunakan linked list.

Selain itu, terdapat beberapa operasi dasar pada queue, yaitu:

1. enqueue, untuk menambahkan elemen ke bagian belakang antrian,
2. dequeue, untuk menghapus dan mengambil elemen dari bagian depan antrian,

3. peek/front, untuk melihat elemen terdepan tanpa menghapusnya,
4. isEmpty, untuk mengecek apakah antrian kosong,
5. size, untuk mengetahui jumlah elemen dalam antrian.

Proses kerja pada queue juga memiliki langkah-langkah tertentu. Pada operasi enqueue, langkahnya adalah:

1. memeriksa apakah queue penuh,
2. jika penuh maka terjadi overflow,
3. jika tidak, posisi rear digeser ke belakang (dalam circular queue menggunakan modulo),
4. elemen baru dimasukkan ke posisi rear.

Sedangkan pada operasi dequeue, langkahnya adalah:

1. memeriksa apakah queue kosong,
2. jika kosong maka terjadi underflow,
3. jika tidak, elemen pada posisi front diambil,
4. posisi front digeser ke depan dan jumlah elemen dikurangi.

Untuk operasi peek/front, langkahnya adalah:

1. memeriksa apakah queue kosong,
2. jika kosong maka tidak ada elemen yang dapat ditampilkan,
3. jika tidak, elemen pada posisi front dikembalikan.

Dalam implementasinya, queue dapat dibuat menggunakan array maupun linked list. Penggunaan array biasanya digunakan untuk queue ukuran tetap dan sering memanfaatkan konsep circular queue agar penggunaan memori lebih efisien, sedangkan linked list lebih cocok untuk queue dinamis. Dengan demikian, struktur data queue sangat berguna dalam pengelolaan data yang membutuhkan mekanisme masuk dan keluar secara berurutan berdasarkan prinsip FIFO (*First In First Out*).

2.2 Kode Program

2.2.1 QueueArray_2511531016

```
package pekan4_2511531016;

public class QueueArray_2511531016 {
int front_1016, rear_1016, size_1016;
int capacity_1016;
int array_1016[];
public QueueArray_2511531016(int capacity) {
this.capacity_1016 = capacity;
front_1016 = this.size_1016 = 0;
rear_1016 = capacity - 1;
array_1016 = new int[this.capacity_1016];
}
boolean isFull_2511531016(QueueArray_2511531016 queue) {
return (queue.size_1016 == queue.capacity_1016);
}
boolean isEmpty(QueueArray_2511531016 queue) {
return (queue.size_1016 == 0);
}
void enqueue(int item) {
if (isFull_2511531016(this))
return;
this.rear_1016 = (this.rear_1016 +1) % this.capacity_1016;
this.array_1016[this.rear_1016]= item;
this.size_1016 = this.size_1016 + 1;
System.out.println(item + "enqueue to queue");
}

int dequeue_1016() {
if (isEmpty(this))
return Integer.MIN_VALUE;
int item = this.array_1016[this.front_1016];
this.front_1016 = (this.front_1016 + 1) % this.capacity_1016;
this.size_1016 = this.size_1016 - 1;
return item;
}
int front_1016() {
if (isEmpty(this))
return Integer.MIN_VALUE;
return this.array_1016[this.front_1016];
}
int rear_1016() {
if (isEmpty(this))
return Integer.MIN_VALUE;
return this.array_1016[this.rear_1016];
}
// Mencetak elemen antrian
void display_1016() {
int i;
if (front_1016 == rear_1016) {
System.out.printf("\nAntrian Kosong\n");
return;
```

```
}
// Kunjungi dari belakang dan cetak
for (i = front_1016; i < rear_1016; i++) {
System.out.printf(" %d <-- ", array_1016[i]);
}
return;
}
}
```

Kode Program 2.1 (Kode Program QueueArray)

2.2.2 QueueArrayDriver_2511531016

```
package pekan4_2511531016;

public class QueueArrayDriver_2511531016 {
public static void main(String[] args) {
QueueArray_2511531016 queue = new QueueArray_2511531016(1000);
queue.enqueue(10);
queue.enqueue(20);
queue.enqueue(30);
queue.enqueue(40);
System.out.println("Item di depan " + queue.front_1016());
System.out.println("Item paling belakang " + queue.rear_1016());
System.out.println("tampilan queue");
queue.display_1016();
System.out.println();
System.out.println(queue.dequeue_1016() + " dihapus dari queue");
System.out.println("Item di depan " + queue.front_1016());
System.out.println("Item paling belakang " + queue.rear_1016());
System.out.println("tampilan queue setelah satu data dihapus");
queue.display_1016();
}
}
```

Kode Program 2.2 (Kode Program QueueArrayDriver)

2.2.3 QueueLinkedList_2511531016

```
package pekan4_2511531016;
import java.util.Queue;
import java.util.LinkedList;

public class QueueLinkedList_2511531016 {
public static void main(String[] args) {
Queue<Integer> q_1016 = new LinkedList<>();
// Tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
for (int i = 0; i < 6; i++)
q_1016.add(i);
// Menampilkan isi antrian
System.out.println("Elemen Antrian " + q_1016);
```

```
// Untuk menghapus kepala antrian
int hapus_1016 = q_1016.remove();
System.out.println("Hapus Elemen = " + hapus_1016);
System.out.println(q_1016);
// Untuk melihat antrian terdepan
int depan_1016 = q_1016.peek();
System.out.println("Kepala Antrian = " + depan_1016);

int banyak_1016 = q_1016.size();
System.out.println("Size Antrian = " + banyak_1016);
}
}
```

Kode Program 2.3 (Kode Program QueueLinkedList)

2.2.4 ReverseData_2511531016

```
package pekan4_2511531016;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511531016 {
public static void main(String[] args) {
Queue<Integer> q_1016 = new LinkedList<>();
q_1016.add(1);
q_1016.add(2);
q_1016.add(3); // [1, 2, 3]
System.out.println("Sebelum reverse" + q_1016);
Stack<Integer> s_1016 = new Stack<Integer>();
while (!q_1016.isEmpty()) { // Q -> S
s_1016.push(q_1016.remove());
}
while (!s_1016.isEmpty()) { // S -> Q
q_1016.add(s_1016.pop());
}
System.out.println("Sesudah reverse= " + q_1016); // [3, 2, 1]
}
}
```

Kode Program 2.4 (Kode Program ReverseData)

2.2.5 IterasiQueue_2511531016

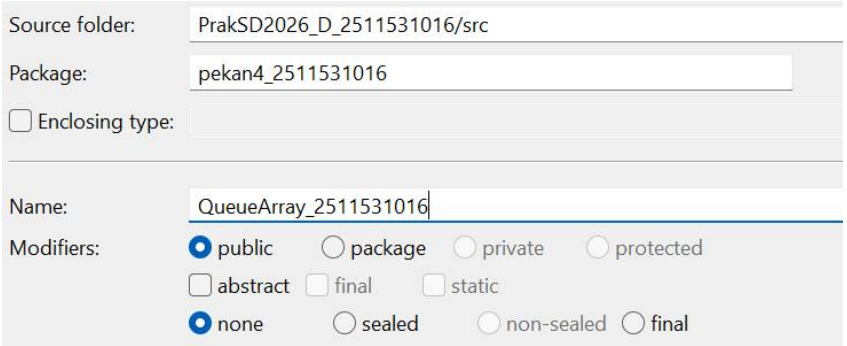
```
package pekan4_2511531016;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class IterasiQueue_2511531016 {
public static void main(String[] args) {
Queue<String> q_1016 = new LinkedList<>();
q_1016.add("Praktikum");
q_1016.add("Struktur");
q_1016.add("Data");
q_1016.add("Dan");
q_1016.add("Algoritma");
Iterator<String> iterator = q_1016.iterator();
while (iterator.hasNext()) {
System.out.print(iterator.next() + " ");
}
}
}
```

Kode Program 2.5 (Kode Program IterasiQueue)

2.3 Langkah Kerja

2.3.1 QueueArray_2511531016

1. Membuat kelas baru bernama *QueueArray_2511531016* dalam *package pekan4_2511531016.*



Gambar 2.1 (Langkah 1 QueueArray)

2. Program diawali dengan deklarasi package dan pembuatan class. Lalu, definisikan variabel yang digunakan untuk mengatur antrian seperti posisi depan, belakang, jumlah data, kapasitas, dan array penyimpanan.

```
package pekan4_2511531016;

public class QueueArray_2511531016 {
int front_1016, rear_1016, size_1016;
int capacity_1016;
int array_1016[];
```

Gambar 2.2 (Langkah 2 QueueArray)

3. Kemudian, Konstruktor digunakan untuk menginisialisasi nilai awal queue, dimana front dan size bernilai 0, rear bernilai capacity - 1, serta array dibuat sesuai kapasitas.

```
public QueueArray_2511531016(int capacity) {
this.capacity_1016 = capacity;
front_1016 = this.size_1016 = 0;
rear_1016 = capacity - 1;
```

```
array_1016 = new int[this.capacity_1016];
}
```

Gambar 2.3 (Langkah 3 QueueArray)

4. Method isFull_2511531016() digunakan untuk mengecek apakah antrian sudah penuh.

```
boolean isFull_2511531016(QueueArray_2511531016 queue) {
return (queue.size_1016 == queue.capacity_1016);
}
```

Gambar 2.4 (Langkah 4 QueueArray)

5. Method isEmpty() digunakan untuk mengecek apakah antrian dalam kondisi kosong.

```
boolean isEmpty(QueueArray_2511531016 queue) {
return (queue.size_1016 == 0);
}
```

Gambar 2.5 (Langkah 5 QueueArray)

6. Method enqueue(int item) digunakan untuk menambahkan data ke dalam antrian, dengan terlebih dahulu mengecek apakah antrian penuh.

```
void enqueue(int item) {
if (isFull_2511531016(this))
return;
```

Gambar 2.6 (Langkah 6 QueueArray)

7. Jika tidak penuh, maka posisi rear digeser menggunakan konsep circular queue.

```
this.rear_1016 = (this.rear_1016 +1) %
this.capacity_1016;System.out.print(" "+ a_1016[i]);
```

Gambar 2.7 (Langkah 7 QueueArray)

8. Selanjutnya, data dimasukkan ke dalam array dan ukuran antrian ditambah, lalu tampilkan pesan bahwa data berhasil ditambahkan.

```
this.array_1016[this.rear_1016]= item;
this.size_1016 = this.size_1016 + 1;
System.out.println(item + "enqueue to queue");
}
```

Gambar 2.8 (Langkah 8 QueueArray)

9. Kemudian, method dequque_1016() digunakan untuk mengambil atau menghapus data dari antrian, dengan mengecek apakah antrian kosong terlebih dahulu.

```
int dequeue_1016() {
if (isEmpty(this))
return Integer.MIN_VALUE;
```

Gambar 2.9 (Langkah 9 QueueArray)

10. Jika tidak kosong, maka data diambil dari posisi front, kemudian front digeser dan size dikurangi 1.

```
int item = this.array_1016[this.front_1016];
this.front_1016 = (this.front_1016 + 1) % this.capacity_1016;
this.size_1016 = this.size_1016 - 1;
return item;
}
```

Gambar 2.10 (Langkah 10 QueueArray)

11. Method front_1016() digunakan untuk melihat elemen paling depan tanpa menghapusnya.

```
int front_1016() {
if (isEmpty(this))
return Integer.MIN_VALUE;
return this.array_1016[this.front_1016];
}
```

Gambar 2.11 (Langkah 11 QueueArray)

12. Method rear_1016() digunakan untuk melihat elemen paling belakang dalam antrian.

```
int rear_1016() {
if (isEmpty(this))
return Integer.MIN_VALUE;
return this.array_1016[this.rear_1016];
}
```

Gambar 2.12 (Langkah 12 QueueArray)

13. Method display_1016() digunakan untuk menapilkan seluruh isi antrian dari depan kebelakang.

```
// Mencetak elemen antrian
void display_1016() {
int i;
if (front_1016 == rear_1016) {
System.out.printf("\nAntrian Kosong\n");
return;
}
```

Gambar 2.13 (Langkah 13 QueueArray)

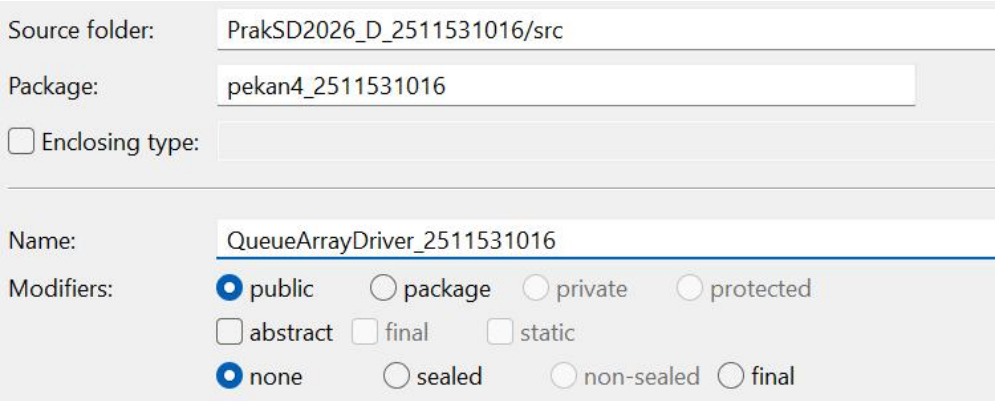
14. Terakhir, lakukan perulangan untuk mencetak elemen antrian satu persatu.

```
// Kunjungi dari belakang dan cetak
for (i = front_1016; i < rear_1016; i++) {
System.out.printf(" %d <-- ", array_1016[i]);
}
return;
}
```

Gambar 2.14 (Langkah 14 QueueArray)

2.3.2 QueueArrayDriver_2511531016

1. Membuat kelas baru bernama *QueueArrayDriver_2511531016* dalam *package pekan4_2511531016.*



Gambar 2.15 (Langkah 1 QueueArrayDriver)

2. Deklarasikan package dan class. Lalu, buat program objek antrian dengan kapasitas 1000 menggunakan class QueueArray_2511531016.

```
package pekan4_2511531016;

public class QueueArrayDriver_2511531016 {
public static void main(String[] args) {
QueueArray_2511531016 queue = new QueueArray_2511531016(1000);
```

Gambar 2.16 (Langkah 2 QueueArrayDriver)

3. Selanjutnya, tambahkan beberapa data ke dalam antrian menggunakan method enqueue, yaitu 10, 20, 30, dan 40.

```
queue.enqueue(10);
queue.enqueue(20);
queue.enqueue(30);
queue.enqueue(40);
```

Gambar 2.17 (Langkah 3 QueueArrayDriver)

4. Kemudian, tampilkan elemen paling depan dan paling belakang, serta cetak seluruh isi antrian menggunakan method front_1016(), rear_1016(), dan display_1016().

```
System.out.println("Item di depan " + queue.front_1016());
System.out.println("Item paling belakang " + queue.rear_1016());
System.out.println("tampilan queue");
queue.display_1016();
```

Gambar 2.18 (Langkah 4 QueueArrayDriver)

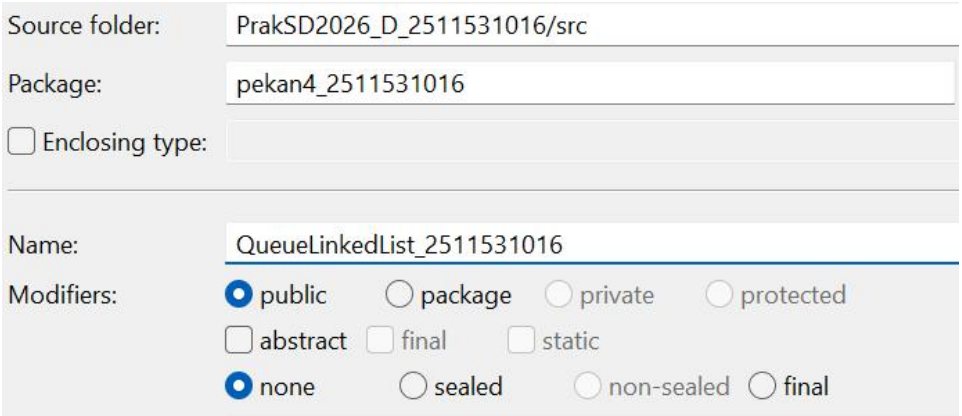
5. Terakhir, hapus satu elemen dari antrian menggunakan dequeue_1016(), lalu kembali tampilkan elemen depan, belakang, dan isi antrian setelah penghapusan.

```
System.out.println();
System.out.println(queue.dequeue_1016() + " dihapus dari queue");
System.out.println("Item di depan " + queue.front_1016());
System.out.println("Item paling belakang " + queue.rear_1016());
System.out.println("tampilan queue setelah satu data dihapus");
queue.display_1016();
}
}
```

Gambar 2.19 (Langkah 5 QueueArrayDriver)

2.3.3 QueueLinkedList_2511531016

1. Membuat kelas baru bernama *QueueLinkedList_2511531016* dalam *package pekan4_2511531016.*



Gambar 2.20 (Langkah 1 QueueLinkedList)

2. Program dimulai dengan deklarasi package, import Queue dan LinkedList, kemudian buat class sebagai pengelola data serta objek antrian menggunakan implementasi LinkedList.

```
package pekan4_2511531016;
import java.util.Queue;
import java.util.LinkedList;
public class QueueLinkedList_2511531016 {
public static void main(String[] args) {
Queue<Integer> q_1016 = new LinkedList<>();
```

Gambar 2.21 (Langkah 2 QueueLinkedList)

3. Selanjutnya, tambahkan elemen ke dalam antrian menggunakan perulangan for, yaitu dari 0 sampai 5.

```
// Tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
for (int i = 0; i < 6; i++)
q_1016.add(i);
```

Gambar 2.22 (Langkah 3 QueueLinkedList)

4. Kemudian program menampilkan seluruh isi antrian yang telah diisi.

```
// Menampilkan isi antrian
System.out.println("Elemen Antrian " + q_1016);
```

Gambar 2.23 (Langkah 4 QueueLinkedList)

5. Hapus elemen paling depan dari antrian menggunakan method remove(), lalu tampikan hasilnya.

```
// Untuk menghapus kepala antrian
int hapus_1016 = q_1016.remove();
System.out.println("Hapus Elemen = " + hapus_1016);
System.out.println(q_1016);
```

Gambar 2.24 (Langkah 5 QueueLinkedList)

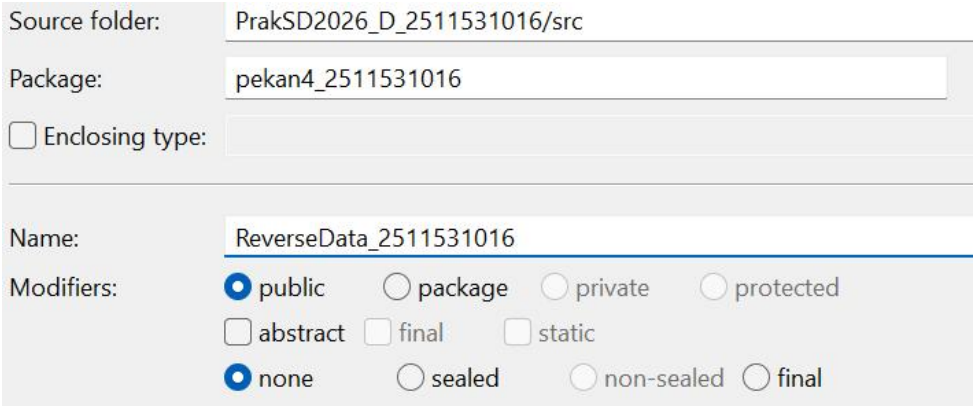
6. Terakhir, lihat elemen terdepan menggunakan peek() tanpa menghapusnya, serta tampilkan jumlah elemen dalam antrian menggunakan size().

```
// Untuk melihat antrian terdepan
int depan_1016 = q_1016.peek();
System.out.println("Kepala Antrian = " + depan_1016);

int banyak_1016 = q_1016.size();
System.out.println("Size Antrian = " + banyak_1016);
}
}
```

Gambar 2.25 (Langkah 6 QueueLinkedList)

2.3.4 ReverseData_2511531016

1. Membuat kelas baru bernama *ReverseData_2511531016* dalam *package pekan4_2511531016.*



Gambar 2.26 (Langkah 1 ReverseData)

2. Program dimulai dengan deklarasi package, import LinkedList, Queue, dan Stack, kemudian buat class utama.

```
package pekan4_2511531016;
import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511531016 {
```

Gambar 2.27 (Langkah 2 ReverseData)

3. Selanjutnya, buat objek queue dan tambahkan beberapa elemen awal. Lalu, tampilkan isi antrian sebelum melakukan proses reverse.

```
Queue<Integer> q_1016 = new LinkedList<>();
q_1016.add(1);
q_1016.add(2);
q_1016.add(3); // [1, 2, 3]
System.out.println("Sebelum reverse" + q_1016);
```

Gambar 2.28 (Langkah 3 ReverseData)

4. Kemudian, buat objek stack dan pindahkan seluruh elemen dari queue ke stack menggunakan perulangan while (Q --> S).

```
Stack<Integer> s_1016 = new Stack<Integer>();
while (!q_1016.isEmpty()) { // Q -> S
s_1016.push(q_1016.remove());
}
```

Gambar 2.29 (Langkah 4 ReverseData)

5. Setelah itu, elemen dari stack dikembalikan lagi ke queue menggunakan perulangan while (S --> Q), sehingga urutan menjadi terbalik.

```
while (!s_1016.isEmpty()) { // S -> Q
q_1016.add(s_1016.pop());
}
```

Gambar 2.30 (Langkah 5 ReverseData)

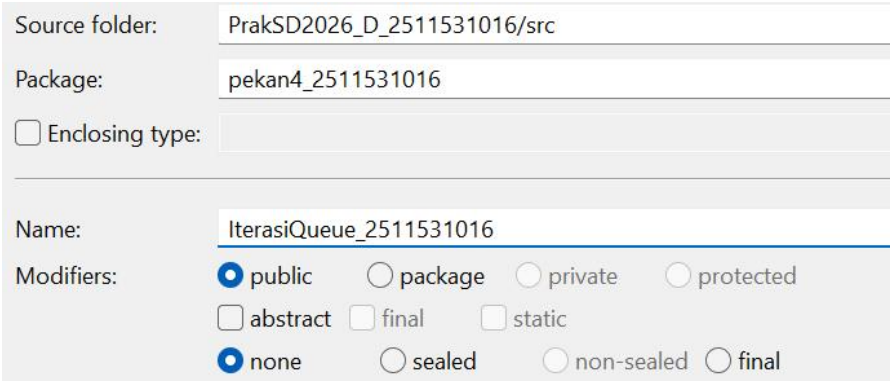
6. Terakhir, tampilkan isi antrian setelah proses reverse, yang menunjukkan urutan elemen sudah berubah.

```
System.out.println("Sesudah reverse= " + q_1016); // [3, 2, 1]
}
}
```

Gambar 2.31 (Langkah 6 ReverseData)

2.3.5 IterasiQueue_2511531016

1. Membuat kelas baru bernama *IterasiQueue_2511531016* dalam *package pekan4_2511531016.*



Gambar 2.32 (Langkah 1 IterasiQueue)

2. Program dimulai dengan deklarasi package, import Iterator, LinkedList, an Queue, serta pembuatan class.

```
package pekan4_2511531016;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.Queue;

public class IterasiQueue_2511531016 {
```

Gambar 2.33 (Langkah 2 IterasiQueue)

3. Kemudian, buat objek antrian bertipe String dan tambahkan beberapa elemen ke dalamnya.

```
public static void main(String[] args) {
Queue<String> q_1016 = new LinkedList<>();
q_1016.add("Praktikum");
q_1016.add("Struktur");
q_1016.add("Data");
q_1016.add("Dan");
q_1016.add("Algoritma");
```

Gambar 2.34 (Langkah 3 IterasiQueue)

4. Lalu, Buat objek Iterator yang digunakan untuk mengakses elemen-elemen dalam antrian secara berurutan.

```
Iterator<String> iterator = q_1016.iterator();
```

Gambar 2.35 (Langkah 4 IterasiQueue)

5. Lakukan perulangan menggunakan while(iterator.hasNext()) untuk mengecek apakah masih ada elemen yang bisa diakses.

```
while (iterator.hasNext()) {
```

Gambar 2.36 (Langkah 5 IterasiQueue)

6. Setiap elemen dalam antrian ditampilkan satu per satu menggunakan iterator.next() hingga semua elemen selesai dicetak.

```
System.out.print(iterator.next() + " ");
}
}
}
```

Gambar 2.37 (Langkah 6 IterasiQueue)

2.4 Analisis Hasil

Berdasarkan hasil praktikum yang telah dilakukan, dapat dilihat bahwa implementasi queue baik menggunakan array maupun *LinkedList* mampu menjalankan operasi dasar seperti *enqueue*, *dequeue*, *peek*, dan *size* dengan baik sesuai prinsip FIFO (*First In First Out*). Pada implementasi array, proses penambahan dan penghapusan elemen terlihat dari perubahan posisi *front* dan *rear*, meskipun masih terdapat keterbatasan pada method display yang belum menampilkan seluruh isi antrian. Sementara itu, penggunaan *LinkedList* memberikan fleksibilitas yang lebih tinggi karena tidak bergantung pada kapasitas tetap, sehingga pengelolaan data menjadi lebih sederhana dengan bantuan method bawaan yang tersedia.

Selain itu, praktikum juga menunjukkan bahwa queue dapat dikombinasikan dengan struktur data lain seperti *stack* untuk melakukan manipulasi data, contohnya dalam proses pembalikan urutan elemen. Penggunaan iterator juga memperlihatkan bahwa elemen dalam queue dapat diakses secara berurutan tanpa mengubah isi antrian. Secara keseluruhan, hasil praktikum ini membantu memperkuat pemahaman mengenai cara kerja queue tidak hanya dari sisi konsep, tetapi juga dari sisi implementasi dan penerapannya dalam berbagai kondisi pemrograman.

BAB III
KESIMPULAN

3.1 Hasil Praktikum

3.1.1 QueueArrayDriver_2511531016

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> QueueArrayDriver_2511531016 [Java Application]
10enqueue to queue
20enqueue to queue
30enqueue to queue
40enqueue to queue
Item di depan 10
Item paling belakang 40
tampilan queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
Item di depan 20
Item paling belakang 40
tampilan queue setelah satu data dihapus
 20 <--  30 <--
```

Gambar 3.1 (Hasil Praktikum QueueArrayDriver)

Output di atas terlihat bahwa setiap data yang dimasukkan melalui proses *enqueue* langsung ditampilkan dengan pesan bahwa data tersebut berhasil masuk ke dalam queue, yaitu 10, 20, 30, dan 40 secara berurutan. Setelah itu, program menampilkan elemen paling depan yaitu 10 dan paling belakang yaitu 40, yang menunjukkan bahwa antrian bekerja sesuai prinsip FIFO. Saat isi queue ditampilkan, urutan yang muncul adalah 10, 20, dan 30, yang mengindikasikan bahwa tidak semua elemen tercetak karena cara perulangan pada method display. Selanjutnya, ketika dilakukan proses *dequeue*, elemen paling depan yaitu 10 dihapus dari antrian, sehingga posisi depan berubah menjadi 20 sementara elemen belakang tetap 40. Tampilan akhir menunjukkan isi queue menjadi 20 dan 30, yang menegaskan bahwa proses penghapusan berjalan sesuai konsep FIFO, yaitu data yang pertama masuk akan menjadi yang pertama keluar.

3.1.2 QueueLinkedList_2511531016

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> QueueLinkedList_2511531016 [Java Application]
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus Elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

Gambar 3.2 (Hasil Praktikum QueueLinkedList)

Output di atas terlihat bahwa antrian awal berisi elemen 0 hingga 5 yang dimasukkan secara berurutan. Ketika dilakukan operasi penghapusan menggunakan remove(), elemen pertama yaitu 0 berhasil dikeluarkan dari antrian, sehingga isi queue berubah menjadi [1, 2, 3, 4, 5]. Selanjutnya, saat program menampilkan kepala antrian menggunakan peek(), nilai yang diperoleh adalah 1, yang menunjukkan elemen terdepan setelah penghapusan. Terakhir, jumlah elemen dalam antrian ditampilkan sebanyak 5, yang menandakan bahwa satu elemen telah berhasil dihapus dan ukuran queue telah berkurang sesuai dengan operasi yang dilakukan.

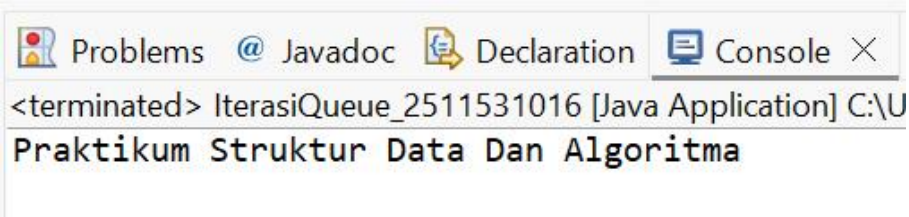
3.1.3 ReverseData_2511531016

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> ReverseData_2511531016 [Java Application] C:\U
Sebelum reverse[1, 2, 3]
Sesudah reverse= [3, 2, 1]
```

Gambar 3.3 (Hasil Praktikum ReverseData)

Output tersebut menunjukkan bahwa sebelum proses dilakukan, queue berisi elemen [1, 2, 3] dengan urutan sesuai data yang dimasukkan. Setelah dilakukan proses *reverse*, urutan elemen berubah menjadi [3, 2, 1], yang menunjukkan bahwa posisi data telah dibalik dari semula. Perubahan ini terjadi karena seluruh elemen dari queue dipindahkan ke dalam stack terlebih dahulu, kemudian dikembalikan lagi ke queue, sehingga urutannya menjadi terbalik. Hal ini menunjukkan bahwa kombinasi penggunaan queue dan stack dapat dimanfaatkan untuk melakukan manipulasi data seperti pembalikan urutan elemen.

3.1.4 IterasiQueue_2511531016



Gambar 3.4 (Hasil Praktikum IterasiQueue)

Output tersebut menunjukkan bahwa setiap elemen dalam queue dicetak satu per satu menggunakan proses iterasi, sehingga menghasilkan urutan "Praktikum Struktur Data Dan Algoritma". Hal ini menunjukkan bahwa iterator berhasil mengakses seluruh elemen antrian secara berurutan dari depan ke belakang tanpa mengubah isi queue tersebut. Proses ini menegaskan bahwa iterasi pada queue dapat digunakan untuk menampilkan atau membaca data tanpa melakukan operasi penambahan maupun penghapusan elemen.

3.2 Saran Pengembangan

Sebagai pengembangan dari praktikum yang telah dilakukan, implementasi queue berbasis array masih bisa diperbaiki, terutama pada bagian method display agar mampu menampilkan seluruh elemen dalam antrian, termasuk elemen terakhir. Selain itu, penanganan kondisi penuh (*overflow*) dan kosong (*underflow*) bisa dibuat lebih informatif, misalnya dengan menampilkan pesan yang jelas atau menggunakan mekanisme penanganan error yang lebih baik, sehingga program tidak hanya berhenti begitu saja tanpa penjelasan.

Pada penggunaan *LinkedList*, meskipun sudah cukup fleksibel, pengembangan bisa diarahkan pada pembuatan fitur tambahan seperti pencarian elemen dalam queue atau penghapusan elemen tertentu tanpa harus selalu dari depan. Sementara itu, pada program reverse dan iterasi, dapat dikembangkan dengan menambahkan variasi data atau membuat fungsi yang lebih umum sehingga dapat digunakan kembali dalam kasus lain. Secara keseluruhan, praktikum ini masih bisa ditingkatkan dengan memperhatikan aspek efisiensi, kejelasan output, serta penambahan fitur yang membuat program lebih interaktif dan mendekati kebutuhan nyata dalam pengolahan data.

DAFTAR PUSTAKA

[1] R. Munir, *Struktur Data dan Algoritma dalam Pemrograman*, edisi revisi. Bandung: Informatika, 2022.

[2] A. Kadir, *Dasar Pemrograman Java dan Struktur Data*, Yogyakarta: Andi, 2021.

[3] B. Raharjo, *Belajar Pemrograman Java untuk Pemula dan Lanjut*, Bandung: Informatika, 2022.

[4] S. Wibowo dan D. Prasetyo, *Algoritma dan Struktur Data: Teori dan Implementasi*, Jakarta: Elex Media Komputindo, 2023.

[5] Oracle, "Queue Interface in Java," *Oracle Documentation*, 2024. [Online]. Available: https://docs.oracle.com/javase/8/docs/api/java/util/Queue.html

TUGAS PRAKTIKUM PEKAN 4

Antrian Pelanggan di Loket Pelayanan

1. Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan struktur data Queue (Antrian) menggunakan array dalam bahasa Java.

Studi kasus yang digunakan adalah antrian pelanggan di loket pelayanan. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip FIFO (First In First Out).

2. Kode Program

2.1 AntrianLoket_2511531016

```java
package pekan4_2511531016;
import java.util.Stack;

public class AntrianLoket_2511531016 {
    String queue_1016[];
    int front_1016, rear_1016, max_1016;

    public AntrianLoket_2511531016(int kapasitas) {
        max_1016 = kapasitas;
        queue_1016 = new String[max_1016];
        front_1016 = -1;
        rear_1016 = -1;
    }

    public boolean isEmpty_1016() {
        return (front_1016 == -1);
    }

    public boolean isFull_1016() {
        return (rear_1016 == max_1016 - 1);
    }

    public void enqueue_1016(String data_1016) {
        if (isFull_1016()) {
            System.out.println("Antrian penuh!");
            return;
        }
        if (isEmpty_1016()) {
            front_1016 = 0;
        }
        rear_1016++;
        queue_1016[rear_1016] = data_1016;
        System.out.println("Data berhasil ditambahkan ke antrian");
    }
```

```java
    public void dequeue_1016() {
        if (isEmpty_1016()) {
            System.out.println("Antrian kosong!");
            return;
        }
        System.out.println(queue_1016[front_1016] + " telah dilayani");
        front_1016++;

        if (front_1016 > rear_1016) {
            front_1016 = rear_1016 = -1;
        }
    }

    public void display_1016() {
        if (isEmpty_1016()) {
            System.out.println("Antrian kosong");
            return;
        }
        System.out.println("Isi antrian:");
        for (int i_1016 = front_1016; i_1016 <= rear_1016; i_1016++) {
            System.out.println((i_1016 - front_1016 + 1) + ". " + queue_1016[i_1016]);
        }
    }

    public void reverse_1016() {
        if (isEmpty_1016()) {
            System.out.println("Antrian kosong");
            return;
        }

        Stack<String> stack_1016 = new Stack<>();

        for (int i_1016 = front_1016; i_1016 <= rear_1016; i_1016++) {
            stack_1016.push(queue_1016[i_1016]);
        }


        int i_1016 = front_1016;
        while (!stack_1016.isEmpty()) {
            queue_1016[i_1016++] = stack_1016.pop();
        }

        System.out.println("Antrian berhasil dibalik");
    }
}
```

Penjelasan :

Class ini berfungsi sebagai inti dari program, karena seluruh mekanisme antrian diatur di dalamnya. Data disimpan dalam array bertipe String, dengan variabel front dan rear untuk menandai posisi depan dan belakang antrian. Saat program dijalankan, antrian diinisialisasi dalam kondisi kosong, lalu elemen akan ditambahkan melalui method enqueue dan dihapus menggunakan dequeue. Alur ini menunjukkan bagaimana konsep FIFO diterapkan, di mana data yang pertama masuk akan menjadi yang pertama diproses.

Selain operasi dasar, class ini juga menyediakan method tambahan seperti display untuk menampilkan isi antrian dan reverse untuk membalik urutan data. Proses reverse dilakukan dengan bantuan stack, yang secara tidak langsung memanfaatkan prinsip LIFO untuk membalik urutan elemen. Dengan adanya method-method ini, class tidak hanya menangani operasi standar queue, tetapi juga menunjukkan bagaimana struktur data bisa dikembangkan untuk kebutuhan yang lebih kompleks.

2.2 Queue_2511531016

```java
package pekan4_2511531016;
import java.util.Scanner;

public class Queue_2511531016 {
    public static void main(String[] args) {
        Scanner input_1016 = new Scanner(System.in);
        AntrianLoket_2511531016 q_1016 = new AntrianLoket_2511531016(10);

        int choice_1016;
        String nama_1016;

        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            choice_1016 = input_1016.nextInt();
            input_1016.nextLine();

            switch (choice_1016) {
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    nama_1016 = input_1016.nextLine();
                    q_1016.enqueue_1016(nama_1016);
                    break;
                case 2:
                    q_1016.dequeue_1016();
                    break;
                case 3:
                    q_1016.display_1016();
                    break;
```

```
                case 4:
                    q_1016.reverse_1016();
                    break;
                case 5:
                    System.out.println("Program selesai");
                    break;
                default:
                    System.out.println("Pilihan tidak valid");
            }
        } while (choice_1016 != 5);
    }
}
```

Penjelasan :

Class ini berperan sebagai pengendali jalannya program melalui method main. Di dalamnya terdapat menu interaktif yang memungkinkan pengguna memilih operasi yang ingin dilakukan, seperti menambah antrian, menghapus antrian, menampilkan isi, hingga melakukan reverse. Setiap pilihan pada menu akan memanggil method yang sesuai dari class AntrianLoket, sehingga terlihat jelas pemisahan antara bagian pengolahan data dan bagian interaksi dengan pengguna.

Dengan adanya menu ini, program menjadi lebih mudah diuji dan dipahami karena pengguna dapat langsung melihat perubahan yang terjadi setelah setiap operasi dilakukan. Selain itu, penggunaan perulangan do-while membuat program terus berjalan sampai pengguna memilih untuk keluar. Secara keseluruhan, class ini melengkapi fungsi dari class sebelumnya dengan menyediakan cara penggunaan yang lebih nyata dan interaktif.

3. Output Kode Program

```
Problems  @ Javadoc  Declaration  Console ×
<terminated> Queue_2511531016 [Java Application] C:\Users\User\.p2\pool
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Kinaya
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: Razel
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Kinaya
2. Razel

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Razel
2. Kinaya

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
Razel telah dilayani
```

```
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. Kinaya

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```

Penjelasan :

Berdasarkan output yang ditampilkan, program berjalan secara interaktif melalui menu yang tersedia. Pada awalnya, pengguna memilih menu untuk menambahkan antrian dan memasukkan nama "Kinaya", kemudian dilanjutkan dengan menambahkan "Razel". Kedua data tersebut berhasil masuk ke dalam antrian secara berurutan. Saat ditampilkan, isi antrian menunjukkan urutan sesuai data masuk, yaitu Kinaya di posisi pertama dan Razel di posisi kedua. Hal ini menunjukkan bahwa sistem antrian berjalan sesuai prinsip FIFO, di mana urutan penyimpanan data tetap terjaga.

Selanjutnya, ketika dilakukan proses *reverse*, urutan antrian berubah menjadi Razel di depan dan Kinaya di belakang, yang menandakan bahwa fitur pembalikan data bekerja dengan baik. Setelah itu, saat dilakukan penghapusan antrian, Razel sebagai elemen terdepan dikeluarkan terlebih dahulu. Ketika antrian ditampilkan kembali, hanya tersisa Kinaya sebagai satu-satunya data. Program kemudian diakhiri dengan memilih menu keluar. Secara keseluruhan, output ini menunjukkan bahwa seluruh fungsi utama seperti penambahan, penampilan, pembalikan, dan penghapusan data telah berjalan sesuai dengan yang diharapkan.